



CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant
Teaching Professor
Lisa Yan

Caches I: Performance (CPI, AMAT)

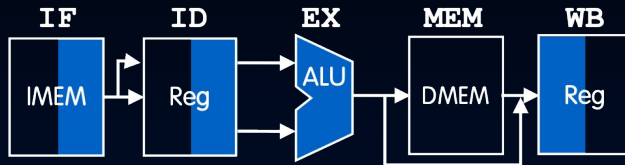


6 Great Ideas in Computer Architecture

1. Abstraction (Layers of Representation/Interpretation)
2. Moore's Law
- 3. Principle of Locality/Memory Hierarchy**
4. Parallelism
- 5. Performance Measurement & Improvement**
6. Dependability via Redundancy

Single-Cycle Datapath

High-level diagram



Stage Timing

$t_{stage} = 200, 100, 200, 200, 100 \text{ ps}$
(Reg stages ID, WB only 100 ps)

Instruction Latency

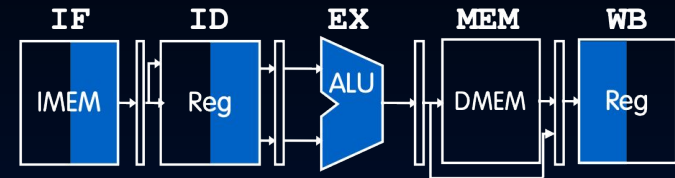
$t_{instruction} = 800 \text{ ps}$

Clock frequency

$1/t_{instruction} = 1/800 \text{ ps}$
 $= 1.25 \text{ GHz}$

throughput gain x4

Pipelined Datapath



$t_{stage} = 200 \text{ ps}$

All stages same length

$t_{instruction} = 5 t_{cycle} = 1000 \text{ ps}$

$1/t_{cycle} = 1/200 \text{ ps}$
 $= 5 \text{ GHz}$

What about program execution time?



"Iron Law" of Processor Performance

- "Iron Law" of Processor Performance

—

- The Memory Hierarchy
- Multi-level Caches
- AMAT: Average Memory Access Time
- [at home] Storage



"Iron Law" of Processor Performance (1/3)

$$\underbrace{\frac{\text{time}}{\text{program}}}_{\text{program execution time}} = \frac{\text{instructions}}{\text{program}} \cdot \frac{\text{cycles}}{\text{instructions}} \cdot \frac{\text{time}}{\text{cycle}}$$

This "iron law" defines **processor performance** on a particular task as **program execution time**.

"Iron Law" of Processor Performance (2/3)

$$\frac{\text{time}}{\text{program}} = \frac{\text{instructions}}{\text{program}} \cdot \frac{\text{cycles}}{\text{instructions}} \cdot \frac{\text{time}}{\text{cycle}}$$

Program benchmark

Determined by:

- Task specification
- Algorithm ($O(N^2)$, $O(N)$, etc.)
- Programming language
- Compiler
- Instruction Set Architecture (ISA)

Cycles Per Instruction

Determined by:

- ISA (CISC vs RISC)
- Processor implementation

Clock period

Determined by:

- Processor implementation (critical path)
- Technology (5nm or 14nm)
- Supply voltage (lower voltage more energy-efficient but reduces transistor speed)

"Iron Law" of Processor Performance (3/3)

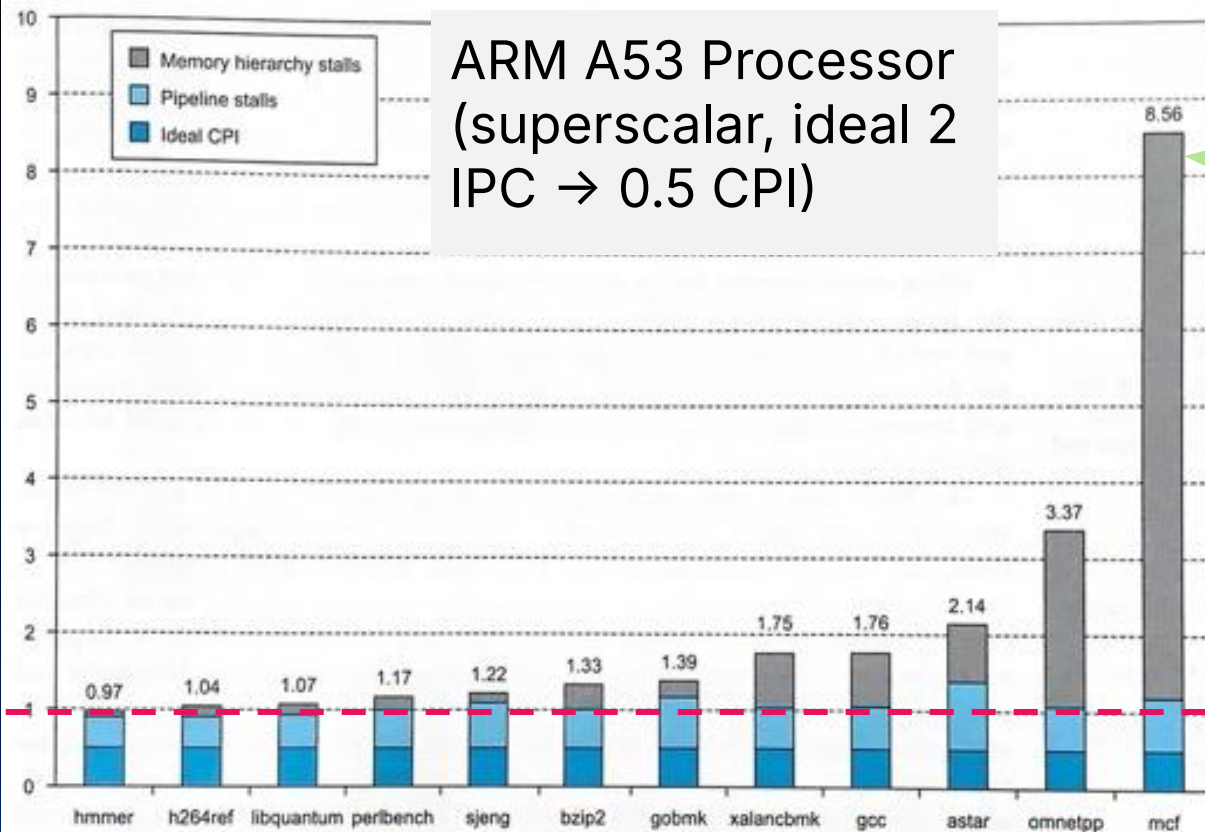
$$\underbrace{\frac{\text{time}}{\text{program}}}_{\text{can measure}} = \underbrace{\frac{\text{instructions}}{\text{program}}}_{\text{can count}} \cdot \underbrace{\frac{\text{cycles}}{\text{instructions}}}_{\text{CPI (average)}} \cdot \underbrace{\frac{\text{time}}{\text{cycle}}}_{1/(\text{clock rate})}$$

To measure (average) CPI, time a processor on a **program benchmark**.

Evaluate one architecture on different benchmarks

ARM A53 Processor
(superscalar, ideal 2
IPC $\rightarrow$ 0.5 CPI)

CPI



NP-hard task:
min cost flow
(see CS170)

CPI = 1



Which Processor Is Faster?

$$\frac{\text{time}}{\text{program}} = \frac{\text{instructions}}{\text{program}} \cdot \frac{\text{cycles}}{\text{instructions}} \cdot \frac{\text{time}}{\text{cycle}}$$

For some task (e.g., image compression), suppose:

	Processor A	Processor B
# instructions	1 million	1.5 million
Average CPI	2.5	1
Clock rate, f	2.5 GHz	2 GHz

Which processor, A, or B, will execute this program faster?

L26: Which processor will execute this program faster?

$$\frac{\text{time}}{\text{program}} = \frac{\text{instructions}}{\text{program}} \cdot \frac{\text{cycles}}{\text{instructions}} \cdot \frac{\text{time}}{\text{cycle}}$$

For some task (e.g., image compression), suppose:

	Processor A	Processor B
# instructions	1 million	1.5 million
Average CPI	2.5	1
Clock rate, f	2.5 GHz	2 GHz

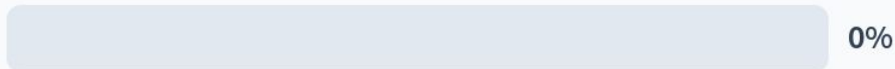
A



B



The same



Don't know



Which Processor Is Faster? Processor B

$$\frac{\text{time}}{\text{program}} = \frac{\text{instructions}}{\text{program}} \cdot \frac{\text{cycles}}{\text{instructions}} \cdot \frac{\text{time}}{\text{cycle}}$$

For some task (e.g., image compression), suppose:

	Processor A	Processor B
# instructions	1 million	1.5 million
Average CPI	2.5	1
Clock rate, f	2.5 GHz	2 GHz
Program execution time	1 ms	0.75 ms ✓

Despite executing more instructions and having a slower clock rate, Processor B is faster for this task!

"Iron Law" of Processor Performance (3/3)

$$\underbrace{\frac{\text{time}}{\text{program}}}_{\text{can measure}} = \underbrace{\frac{\text{instructions}}{\text{program}}}_{\text{can count}} \cdot \underbrace{\frac{\text{cycles}}{\text{instructions}}}_{\text{CPI (average)}} \cdot \underbrace{\frac{\text{time}}{\text{cycle}}}_{1/(\text{clock rate})}$$

To measure (average) CPI, run a processor on a **program benchmark**.

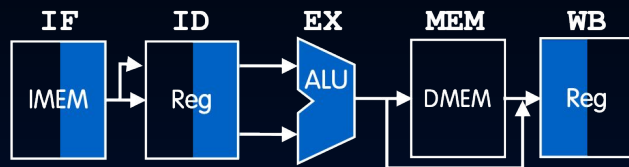
On the same benchmark, processors will have different **CPI** because of differences in **ISA** (CISC vs. RISC) and **processor implementation**:

- CS61C single cycle: CPI = 1
- **Pipelined** processor: CPI ≈ 1
- Complex instructions (CPI >> 1)
- Superscalar: CPI < 1 (multiple instructions completed per cycle on single core)

Compare CPI of Two Architectures

Single-Cycle Datapath

High-level diagram



Clock frequency

$$1/t_{instruction} = 1/800 \text{ ps} = 1.25 \text{ GHz}$$

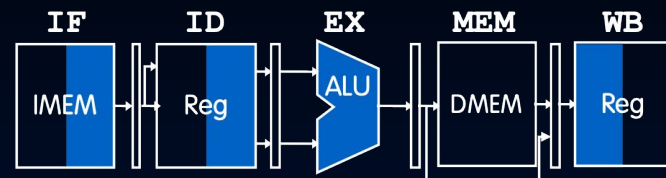
CPI

~1

avg # clock cycles until another completed instruction

throughput gain x4

Pipelined Datapath



$$1/t_{cycle} = 1/200 \text{ ps} = 5 \text{ GHz}$$

~1

(<1 actual; see CS152)

In practice, performance costs due to:

- instruction dependencies/pipeline **hazards**
- **Memory access**



(change of gears)



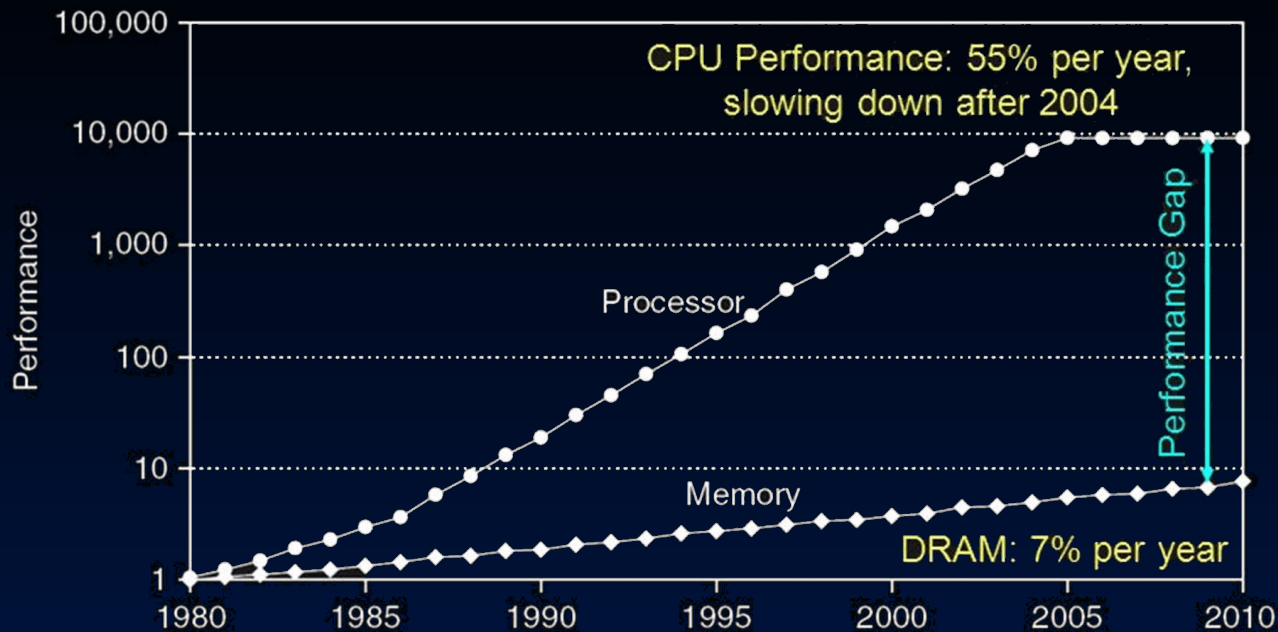
(EVs have no gears)



The Memory Hierarchy

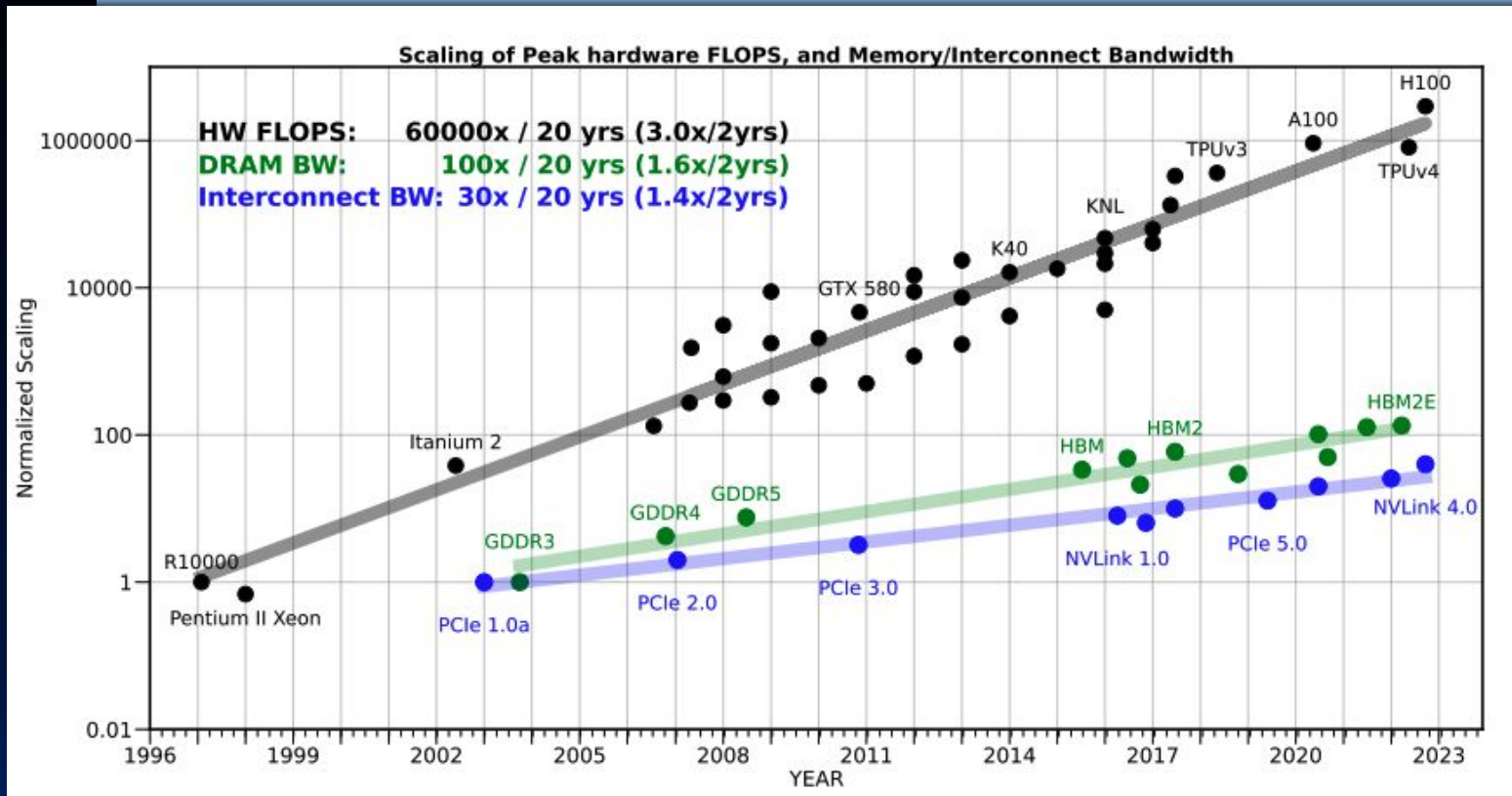
- "Iron Law" of Processor Performance
-
- The Memory Hierarchy
 - Multi-level Caches
 - AMAT: Average Memory Access Time
 - [at home] Storage

Processor-DRAM Gap (Latency), ~2010



1980 microprocessor executes ~one instruction in same time as DRAM access
 2020 microprocessor executes ~1000 instructions in same time as DRAM access
 Slow DRAM access has disastrous impact on CPU performance!

The "Memory Wall" is still active in 2024



"AI and Memory Wall." [arXiv 2024](#)

26-Caches I: Performance (17)

Yan, SP26

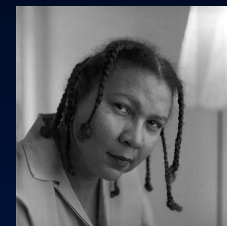
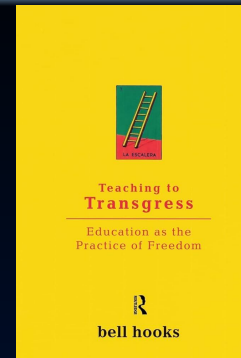
Library Analogy: Why are Large Memories Slow?

- Time to find a book in a large library
 - **Search** a large card catalog – (mapping title/author to index number)
 - **Round-trip time** to check out – walk to the stacks and retrieve the desired book
- Larger libraries worsen both delays!
- Electronic memories have same issue, plus the technologies used to store a bit slow down as density increases (e.g., SRAM vs. DRAM vs. Disk)
- What we want is a **large, yet fast memory**.



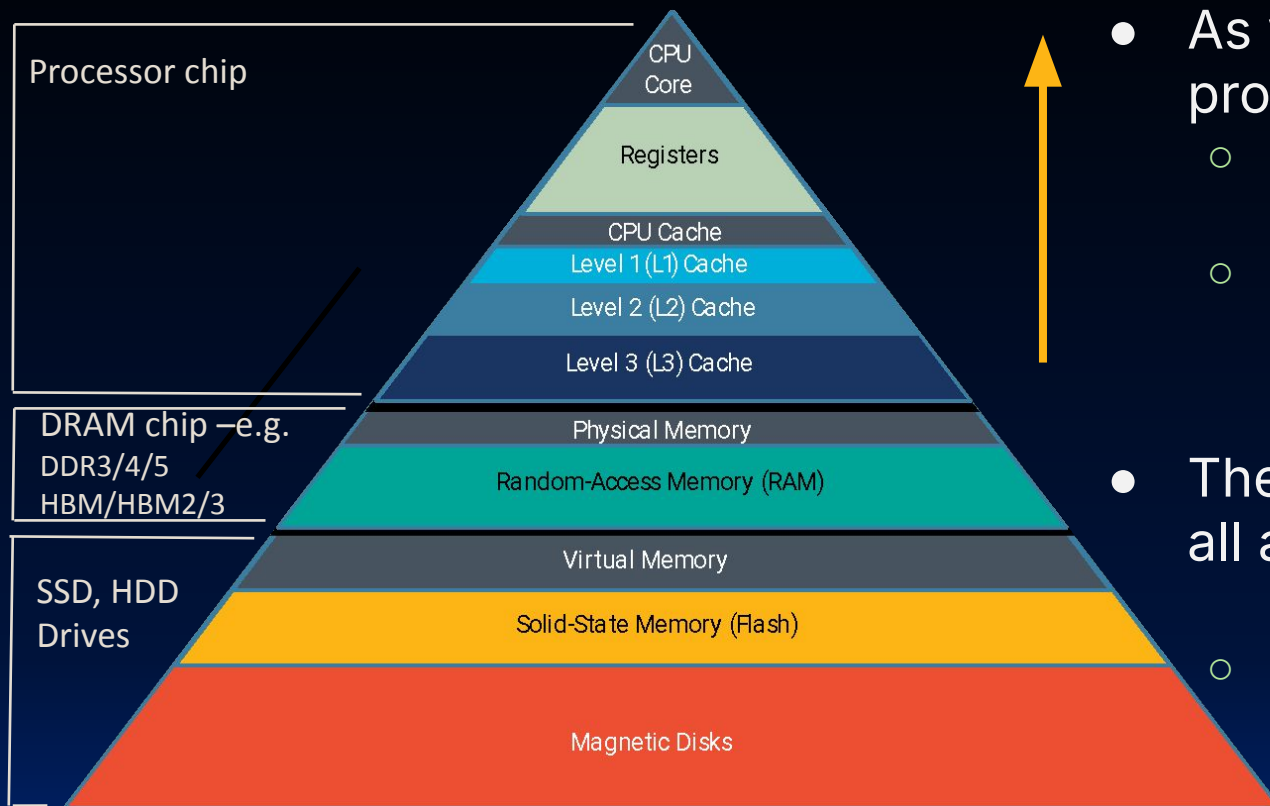
Library Analogy: What to do?

- How do you write a report using library books?
 - (step 0: go to library)
 - Look up a book
 - Check out book from stacks
 - Place book on desk close to library
- If you need more, repeat process to add new book to desk
 - Don't return earlier books just yet, since you might need them (locality)
- Hopefully, this collection of ~10 books on desk enough to write report, despite 10 being only 0.00001% of books in UC Berkeley libraries.



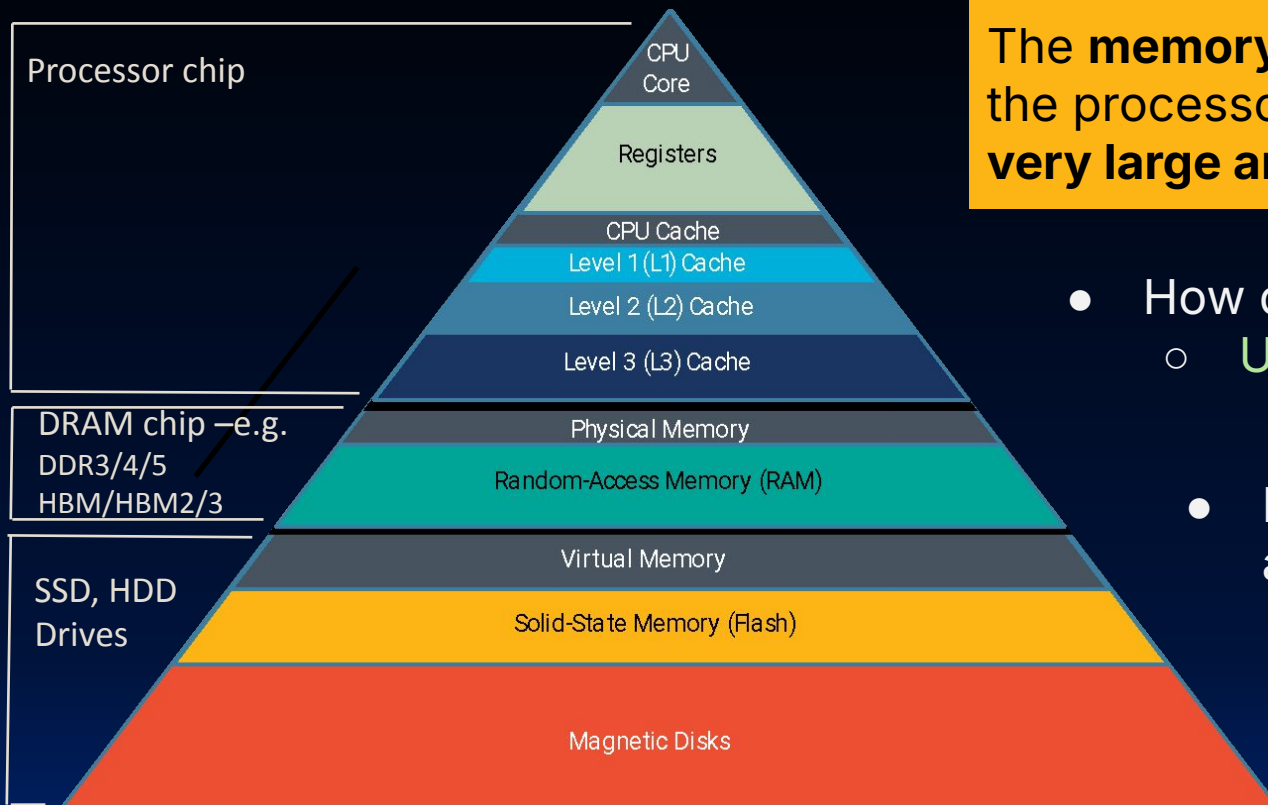
Teaching to Transgress (1994)
bell hooks
1952-2021

Great Idea #3: Principle of Locality / Memory Hierarchy



- As we go closer to the processor:
 - HW is smaller, faster, and more expensive
 - Data is smaller and a **copy** of a subset of data in lower levels.
- The lowest level contains all available data.
 - (Nowadays, we don't go to magnetic disk.)

Great Idea #3: Principle of Locality / Memory Hierarchy

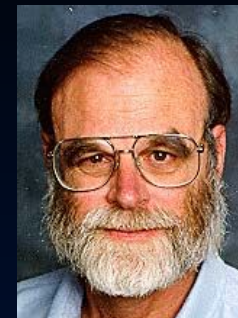
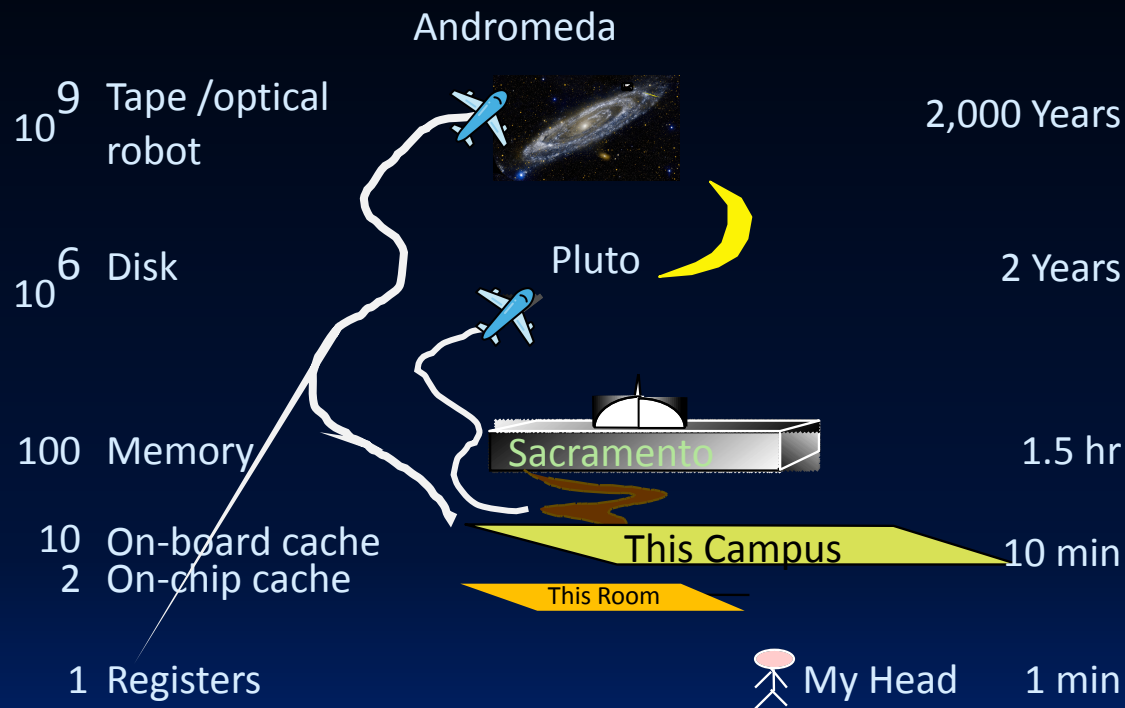


The **memory hierarchy** presents the processor with the illusion of a **very large and fast memory**.

- How do we make it **fast**?
 - Use a **hierarchy**.
- How do we make it appear **large**?
 - **Principle of locality**: Cache the "right" data in higher levels.

Great Idea #3: Principle of Locality / Memory Hierarchy

Storage Latency Analogy: How Far Away is the Data?



Jim Gray
 1998 Turing Award
 B.S. Cal 1966
 Ph.D. Cal 1969

"For seminal contributions to database and transaction processing research and technical leadership in system implementation."

Yan, SP26

Caching and Locality

- A **cache** contains copies of data that are being used.
- A cache works on the principles of **temporal and spatial locality**.

	Temporal Locality	Spatial Locality
Idea	If we use it now, chances are that we'll want to use it again soon.	If we use a piece of memory, chances are we'll use the neighboring pieces soon.
Library Analogy	We keep a book on the desk while we check out another book.	If we check out a book's vol. 1 while we're at it, we'll also check out vol. 2.
Memory	If a memory location is referenced, then it will tend to be referenced again soon . Therefore, keep (copies of) most recently accessed data items closer to the processor.	If a memory location is referenced, the locations with nearby addresses will tend to be referenced soon. Copy lines consisting of contiguous words closer to the processor.

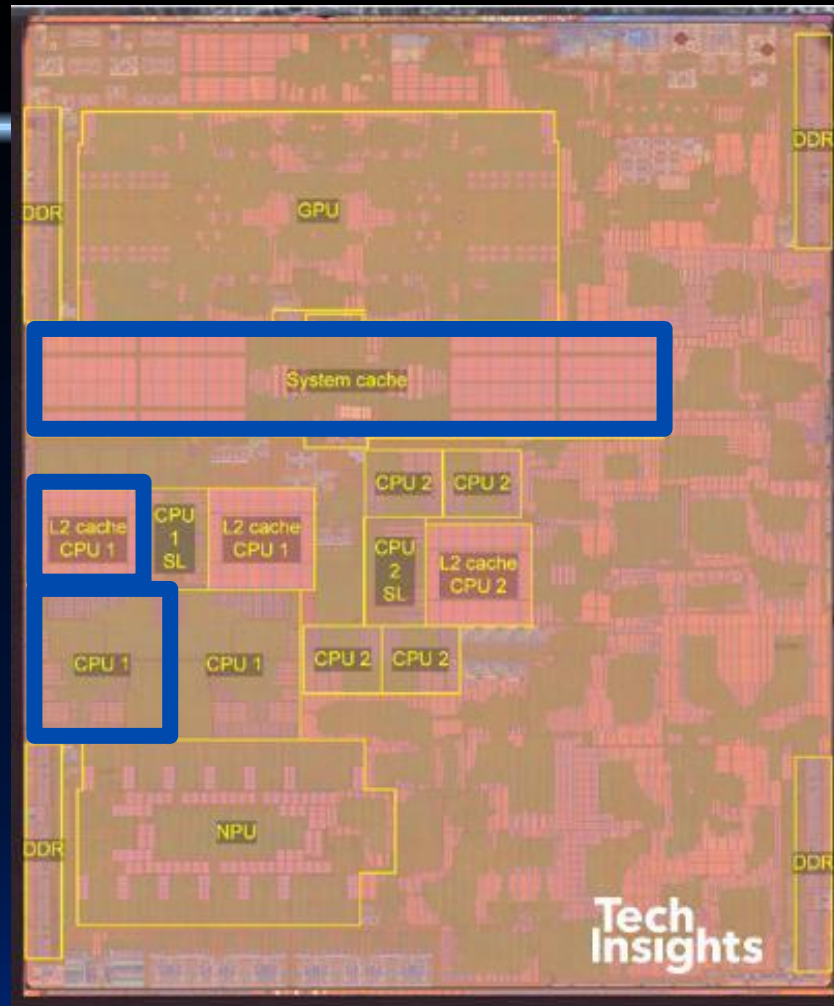


Multi-level Caches

- "Iron Law" of Processor Performance
-
- The Memory Hierarchy
 - Multi-level Caches
 - AMAT: Average Memory Access Time
 - [at home] Storage

Memory Cache

- The “**memory cache**”, or **cache** for short.
 - Usually on the same chip as the CPU.
 - **Faster but more expensive** than DRAM memory.
- The cache is a copy of a subset of main memory.
 - Multiple levels of caches!
 - L1 cache directly on CPU!



Apple A14 Bionic
TechInsights [[source](#)]

26-Caches I: Performance (25)



Demo

- `df -h`
- `df -H`
- `sysctl -a | grep hw.memsize`
- `sysctl -a | grep "hw\.\l.*size"`



```
hw.l1icachesize: 131072
hw.l1dcachesize: 65536
hw.l2cachesize: 4194304
```

Two L1 caches! L1i
(instruction) and L1d (data)

What is the L2 cache size?

- A.** 2 MB
- B.** 2 MiB
- C.** 4 MB
- D.** 4 MiB
- E.** 4 GB
- F.** 4 GiB
- G.** Something else



AMAT: Average Memory Access Time

- "Iron Law" of Processor Performance
-
- The Memory Hierarchy
- Multi-level Caches
- AMAT: Average Memory Access Time
- [at home] Storage



Cache Hit vs. Cache Miss

- **Cache Hit**

- The data you were looking for is in the cache.
- Retrieve the data from the cache and bring it to the processor.

- **Cache Miss**

- The data you were looking for is **not** in the cache.
- Go to a lower layer in the memory hierarchy to find the data, put the data in the cache.
- Then, bring the data to the processor.

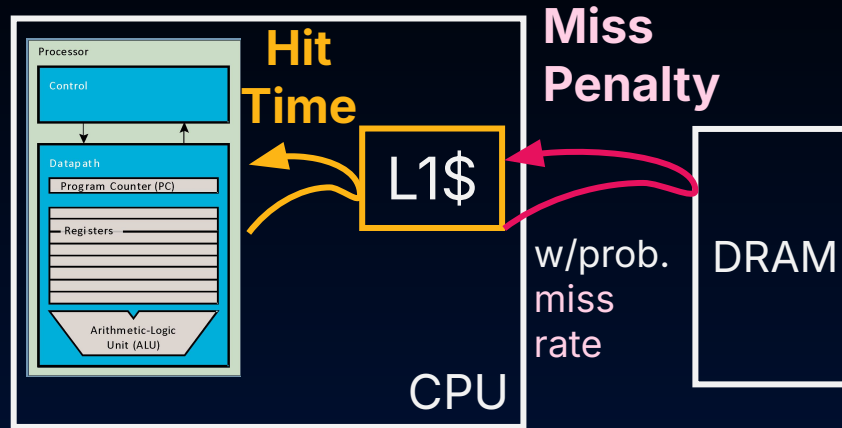
- **Hit rate**: fraction of access that hit in the cache.
- **Hit time**: time (latency) to access cache memory, including determining hit or miss

- **Miss rate**: $1 - \text{Hit rate}$.
- **Miss penalty**: time (latency) to replace a line from lower level in memory hierarchy to cache.

Goal: Design caches to reduce **average memory access time.**



Average Memory Access Time



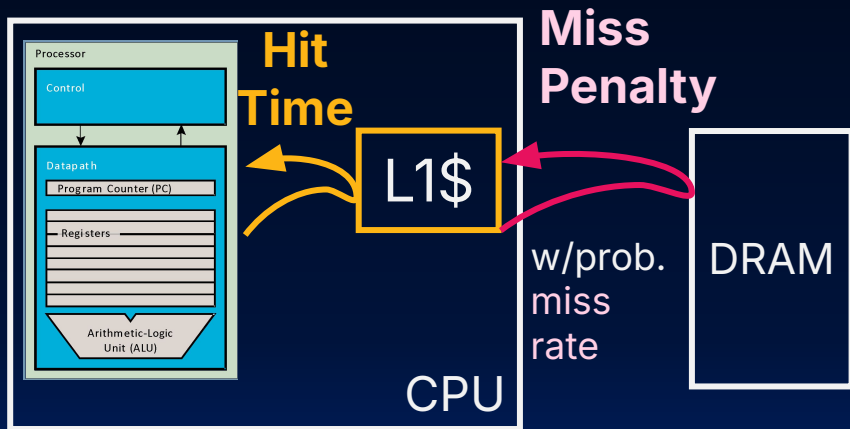
Average Access Memory Time

$$AMAT = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}$$

What is the Average Memory Access Time?

Assume just one L1 cache:

- L1 Hit Time = 1 cycle
- L1 Miss rate = 5%
- L1 Miss penalty = 200 cycles



What is the **Average Memory Access Time**, in # cycles?

- Hint 1: Think probability and expectation!
- Hint 2: On cache miss,
Total Time to retrieve data
= Hit time + Miss penalty.

- A. 1
- B. 1.95
- C. 2
- D. 21
- E. 10.95
- F. 11
- G. Something else

L15: What is the Average Memory Access Time, in cycles?

A. 1

0%

B. 1.95

0%

C. 2

0%

D. 21

0%

E. 10.95

0%

F. 11

0%

G. Something else

0%



What is the Average Memory Access Time?

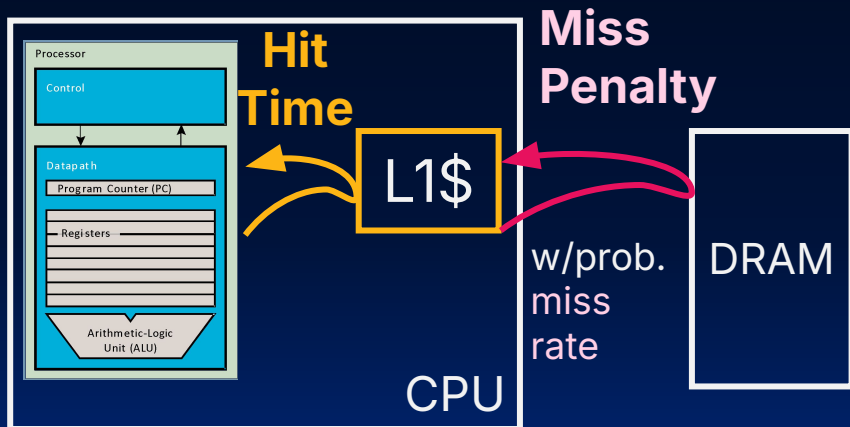
Without L2 cache, assume:

- L1 Hit Time = 1 cycle
- L1 Miss rate = 5%
- L1 Miss penalty = 200 cycles

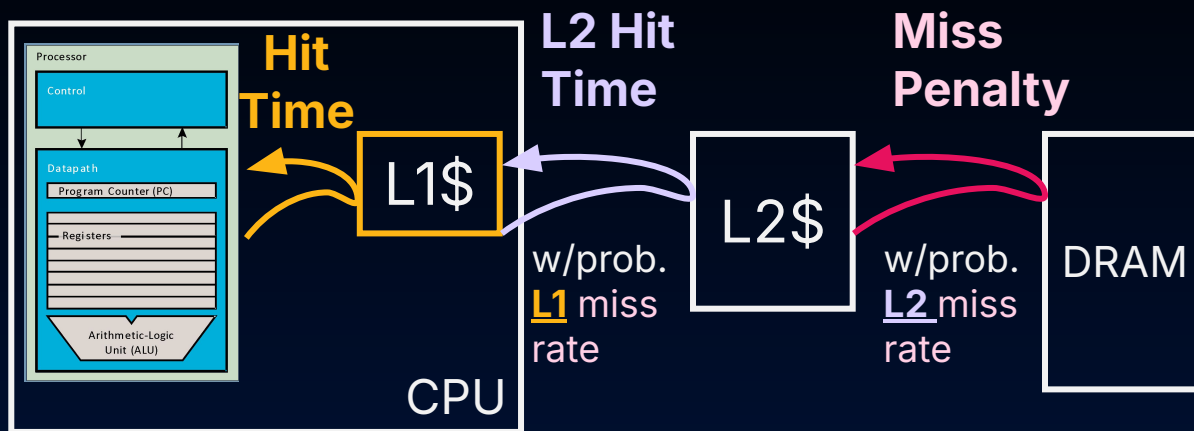
What is the **Average Memory Access Time**, in # cycles?

- Hint 1: Think probability and expectation!
- Hint 2: On cache miss, Total Time to retrieve data = Hit time + Miss penalty.

$$\begin{aligned} AMAT &= \text{Hit Rate} \cdot (\text{Hit Time}) \\ &\quad + \text{Miss Rate} \cdot (\text{Hit Time} + \text{Miss Penalty}) \\ &= \text{Hit Time} + \text{Miss Rate} \cdot (\text{Miss Penalty}) \\ &= (1) + .05(200) = \mathbf{11} \end{aligned}$$



Average Memory Access Time: Recursive Formula



$$AMAT = (L1 \text{ Hit Time}) + L1 \text{ Miss Rate} \cdot (L1 \text{ Avg. Miss Penalty})$$

recursive



$$L2 \text{ Hit Time} + L2 \text{ Miss Rate} \cdot (L2 \text{ Miss Penalty})$$

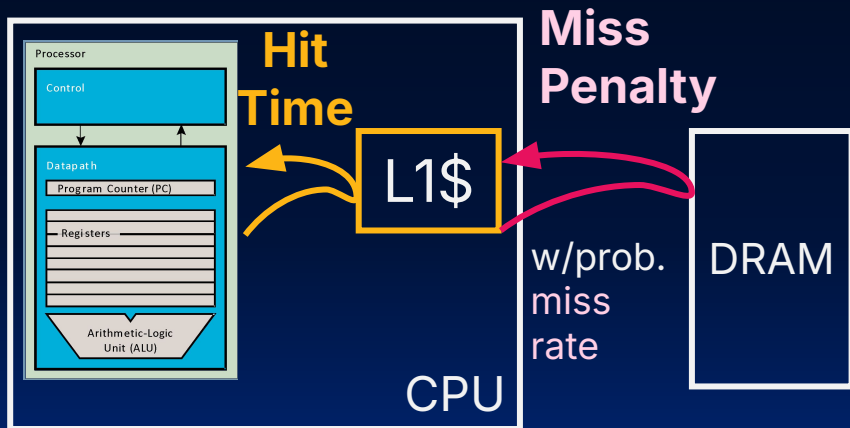
Example 1: Without L2 Cache (from before)

Without L2 cache, assume:

- L1 Hit Time = 1 cycle
- L1 Miss rate = 5%
- L1 Miss penalty = 200 cycles

AMAT

$$\begin{aligned}
 &= \text{L1 hit time} \\
 &\quad + (\text{L1 Miss rate}) * (\text{L1 Miss Penalty}) \\
 &= 1 + 0.05 \times 200 \\
 &= \underline{11 \text{ cycles}}
 \end{aligned}$$



[for next time]



Example 2: With L2 Cache

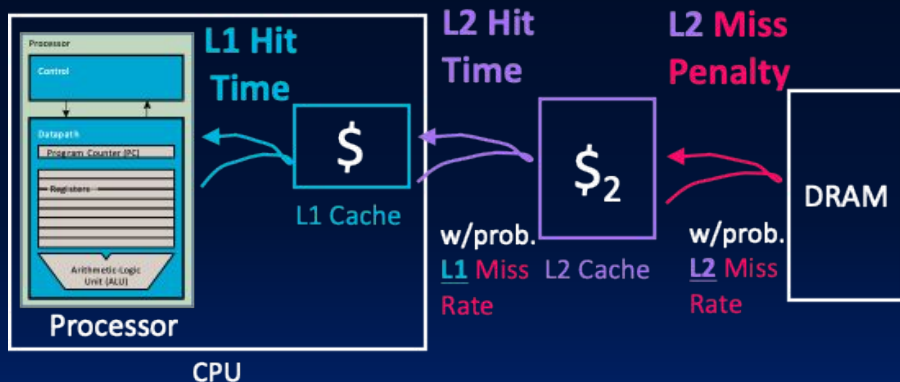
With L2 cache, assume:

- L1 Hit Time = 1 cycle
- L1 Miss rate = 5%
- L2 Hit Time = 5 cycles
- L2 Miss rate = 15%
- L2 Miss penalty = 200 cycles

L1 (average) miss penalty
= L2 hit time
+ (L2 miss rate)*(L2 miss penalty)
= 5 + 0.15 * 200 = 35

AMAT

= L1 hit time
+ (L1 Miss rate)*(L1 miss penalty)
= 1 + 0.05 * 35
= 2.75 cycles



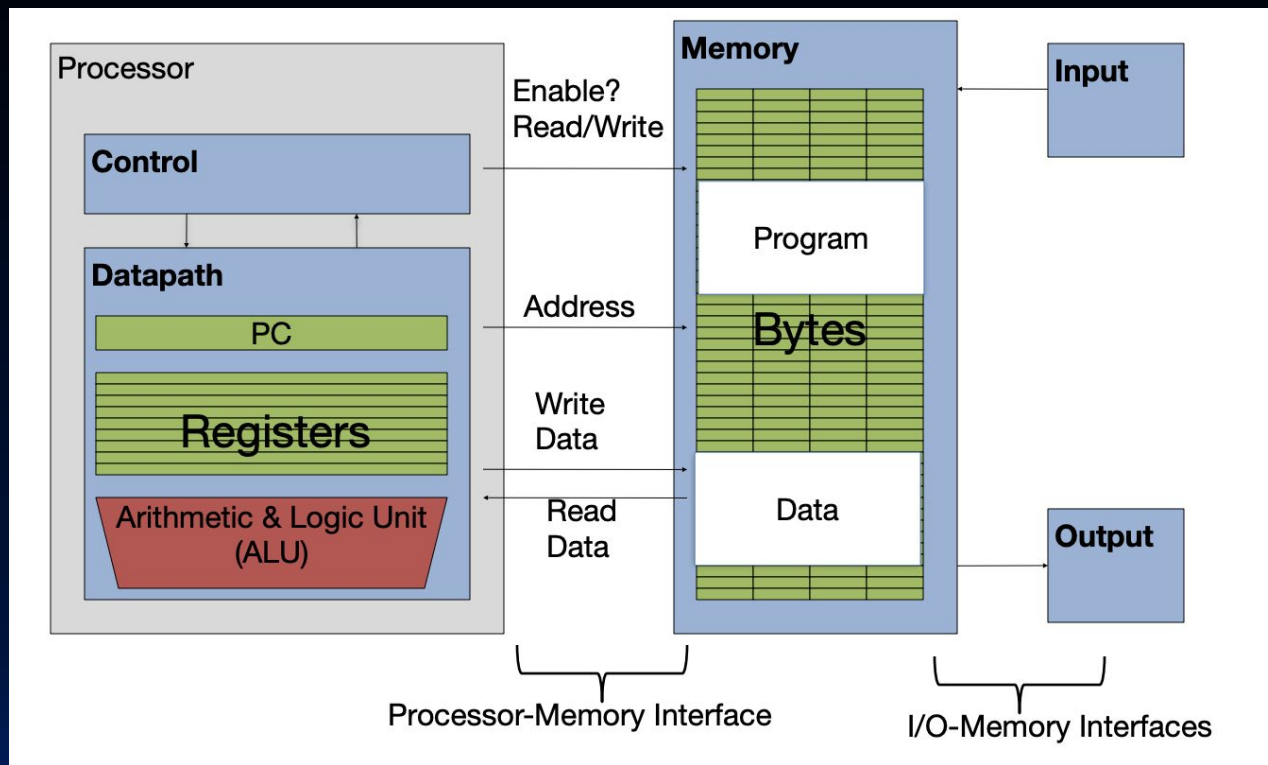
4x faster with L2 cache!
(2.75 vs. 11)



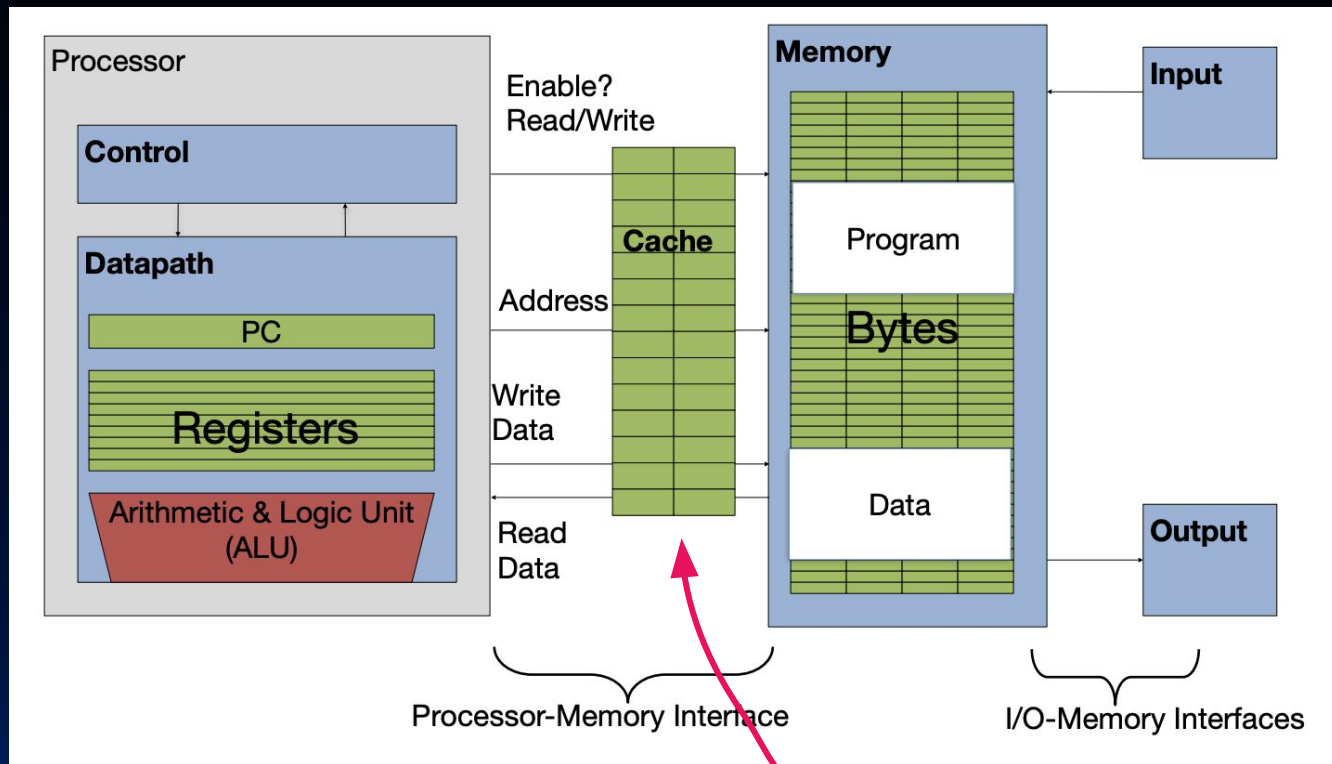
Caches

- "Iron Law" of Processor Performance
-
- The Memory Hierarchy
- Multi-level Caches
- AMAT: Average Memory Access Time
- [at home] Storage

Computer without Cache



Computer with Cache



Lines (i.e., blocks) of data are copied from memory to the cache.



Memory Access without Cache

Load word instruction:

t0

1234

lw t0 0(t1)

t1

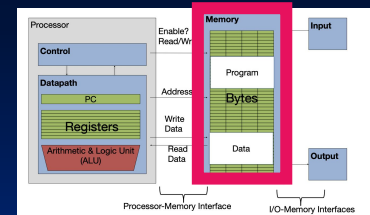
0x12F0

Memory[0x12F0] = 1234

Memory access **without cache**:

1. Processor issues address 0x12F0 to memory
2. **Memory** reads 1234 @ address 0x12F0

3. **Memory** sends 1234 to Processor
4. Processor loads 1234 into register t0



Yan, SP26



Memory Access with Cache

Load word instruction:

t0

1234

Memory[0x12F0] = 1234

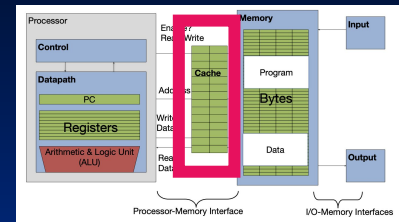
lw t0 0(t1)

t1

0x12F0

Memory access **with cache**:

1. Processor issues address 0x12F0 to **cache**
2. **Cache** checks for copy of data, addr. 0x12F0
 - a. If **hit** (finds match): **cache** reads 1234
 - b. If **miss** (no match): **cache** sends address 0x12F0 to **Memory**
 - i. **Memory** reads line with 1234 @ address 0x12F0
 - ii. **Memory** sends line with 1234 to **cache**
 - iii. **Cache** replaces some line to store 1234, address 0x12F0
3. **Cache** sends 1234 to Processor
4. Processor loads 1234 into register t0



Typical Scale and Reducing Miss Rate

- **L1 cache (L1\$)**
 - size: tens of KB
 - hit time: complete in one clock cycle
 - miss rates: 1-5%
- **L2 cache (L2\$)**
 - size: hundreds of KB
 - hit time: few clock cycles
 - miss rates: 10-20%
- The **L2** miss rate is the fraction of **L1** misses that also miss in **L2**.
 - Why so high? (more later)

Ways to reduce miss rate:

- Larger cache
 - limited by cost and technology
 - Bigger caches slower, whereas hit time of first level cache $<$ cycle time (more later)
- More places in the cache to put each line of memory
 - Cache design! (up next)

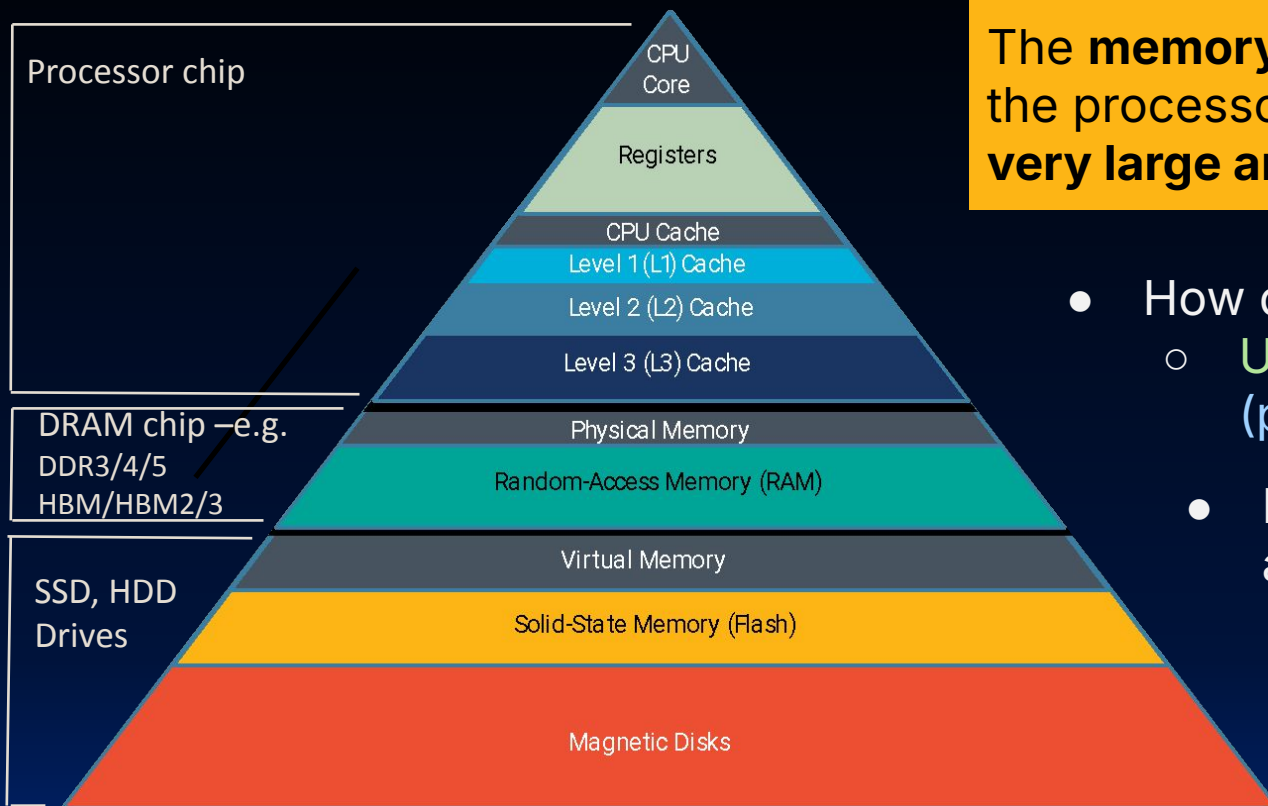


Agenda

[at home] Storage

- "Iron Law" of Processor Performance
-
- The Memory Hierarchy
 - Multi-level Caches
 - AMAT: Average Memory Access Time
 - [at home] Storage

Great Idea #3: Principle of Locality / Memory Hierarchy



The **memory hierarchy** presents the processor with the illusion of a **very large and fast memory**.

- How do we make it **fast**?
 - Use a **hierarchy**. (prev slide)
- How do we make it appear **large**?
 - **Principle of locality**: Cache the "right" data in higher levels.

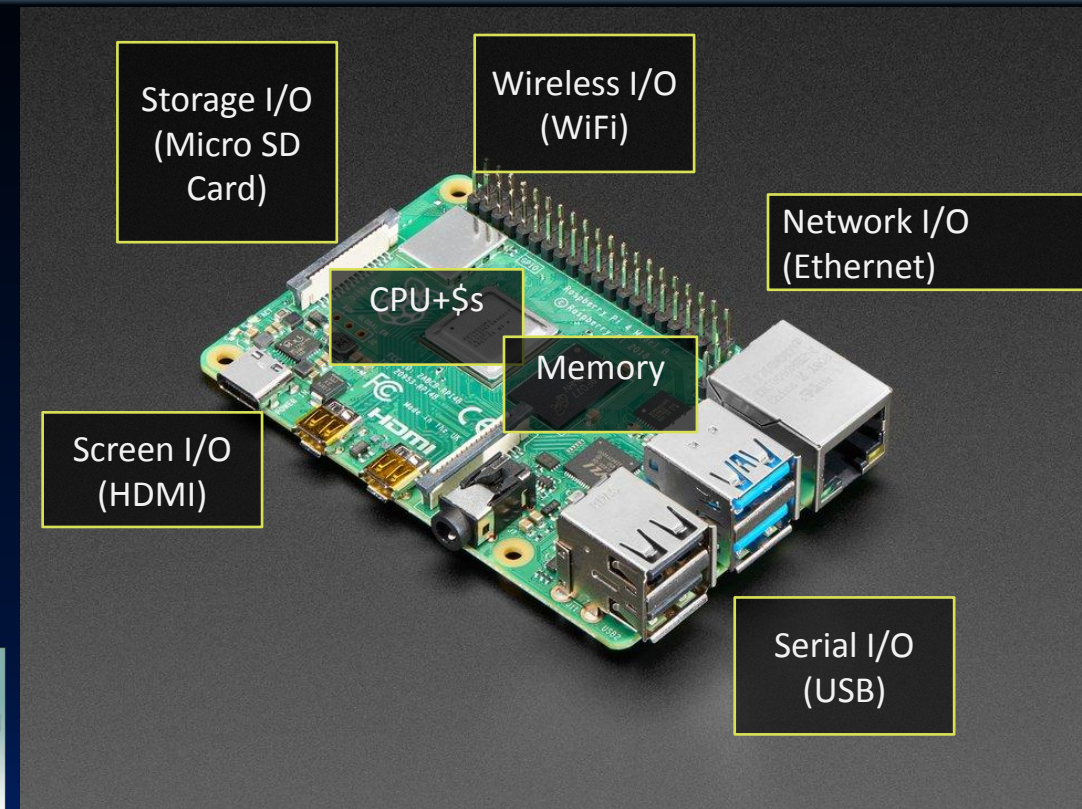
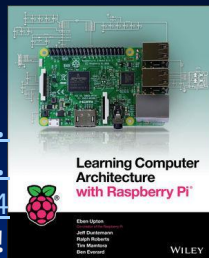
Raspberry Pi (\$35) and I/O ports

The Raspberry Pi is a low-cost computer.

- Motherboard with CPU, I/O, caches, etc., soldered on
- **Cache Memory** on processor (" \$" stands for cache)
- **Main Memory**
- **Disk (Secondary Memory)**

<https://onlinelibrary.wiley.com/doi/book/10.1002/9781119415534>

free w/UCB login!



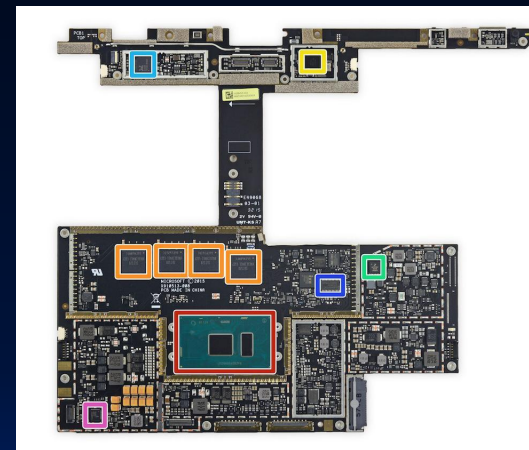
Yan, SP26

Main Memory is DRAM

- Dynamic Random Access Memory:
 - Latency to access first word: $\sim 10\text{ns}$ ($\sim 30\text{-}40$ processor cycles), each successive ($0.5\text{ns} - 1\text{ns}$)
 - Each access brings 64 bits, supports 'bursts'
 - $\$3/\text{GiB}$
- Data is impermanent:
 - **Dynamic**: capacitors store bits, so needs periodic refresh to maintain charge
 - **Volatile**: when power is removed, loses data.
- Contrast with SRAM (for caches):
 - Static (no capacitors) but still volatile
 - Faster (0.5 ns)/more expensive/lower density



Desktop/server
DIMM



MS Surface Book

Storage / "Disk" / Secondary Memory

Attached as a peripheral I/O device. **Non-volatile**

- Solid-State Drive (SSD)
 - Access: 40-100 μ s (~100k proc. cycles)
 - \$0.05-0.5/GB
 - Usually flash memory
- Hard Disk Drive (HDD)
 - Access: <5-10ms (10-20M proc. cycles)
 - \$0.01-0.1/GB
 - Mechanical





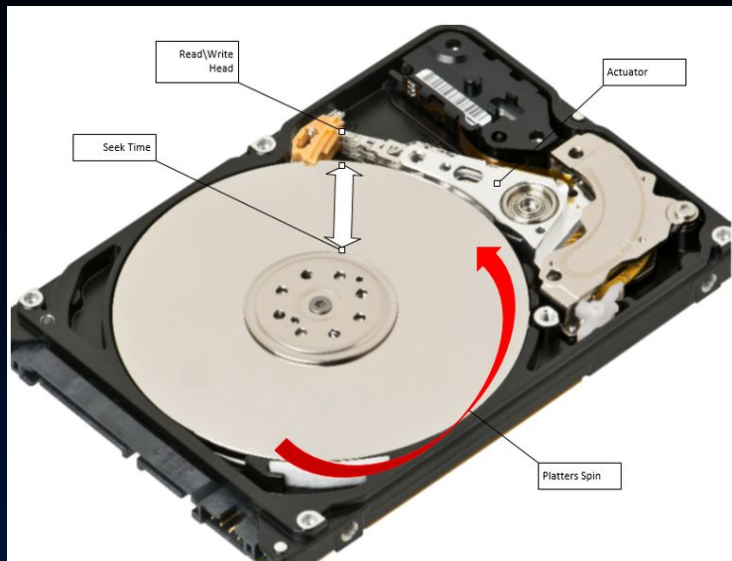
What units do different storage use?

[IEC Prefixes]

- Base-10 (SI): Hard Disk manufacturers & Telecommunications
 - What is advertised as a 1 TB drive actually holds about 90% of what you expect.
 - A 1 Mbit/s connection transfers 10^6 bps.
- Base-2 (IEC): RAM and Caches, for the most part
 - We will try to be clear in this course

RAM and Caches use powers of 2.

Aside ... How do HDDs work?



- 10,000 rpm (revolutions per minute)
- 6 ms per revolution
- Average random access time: 3 ms
- (~107 processor cycles)

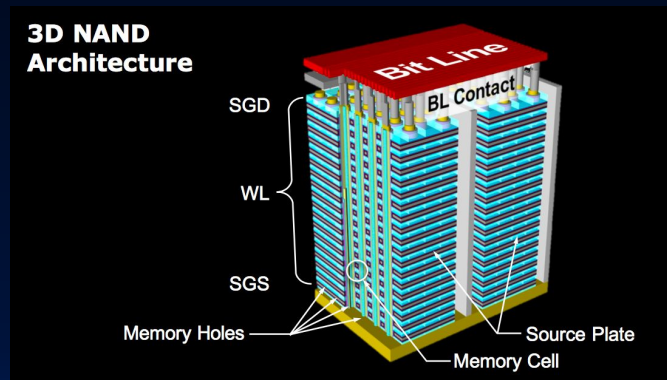
Nick Parlante (Stanford University)
explanation:



[video](#)

Aside 2...How do SSDs work?

- Made with transistors (nothing mechanical/rotating)
- Operates like a "Ginormous" register file.
 - Furthermore, does not "forget" when power is off (non-volatile)
- Fast access to all locations, regardless of address
 - Still much slower than register, DRAM.
 - Read/write blocks, not bytes.
 - Potential reliability issues...
- Some unusual requirements:
 - Can't erase single bits—only entire blocks



Flash memory is generally a 3D array of bit cells (up to 256 layers!)

Yan, SP26

What units do different storage use?

[bonus]

- What about OS?
 - Uses whatever makes sense for marketing (mostly base-10 GB nowadays)

Capacity:	644,420,202,496 bytes	600 GB
On This Computer		
	Computer	3.0 GB / 234.7 GB available /
	Windows7_OS	16.7 GB / 242.4 GB available /dev/sda3 ▲

Capacity:	330.6 GiB
Usage:	36.7 GiB used (11%) 293.9 GiB free (89%)

Capacity:	494.38 GB
Available:	60.68 GB (42.82 GB purgeable)
Used:	459,189,743,616 bytes (459.19 GB on disk)

JOHN DINAN; et al., Plaintiffs-Appellants,

v.

SANDISK LLC, Defendant-Appellee.

[No. 20-15287.](#)

United States Court of Appeals, Ninth Circuit.

Argued and Submitted February 4, 2021 — San Francisco, California.
Filed February 10, 2021.

Plaintiffs appeal the district court's Rule 12(b)(6) dismissal of their complaint against SanDisk. Plaintiffs allege that SanDisk violated three California consumer protection statutes^[1] by using gigabyte ("GB") to mean 1,000,000,000 bytes ("the decimal definition"), when Plaintiffs assumed gigabyte as used by SanDisk meant 1,073,741,824 bytes ("the binary definition"). Plaintiffs additionally allege a breach of contract on similar grounds.